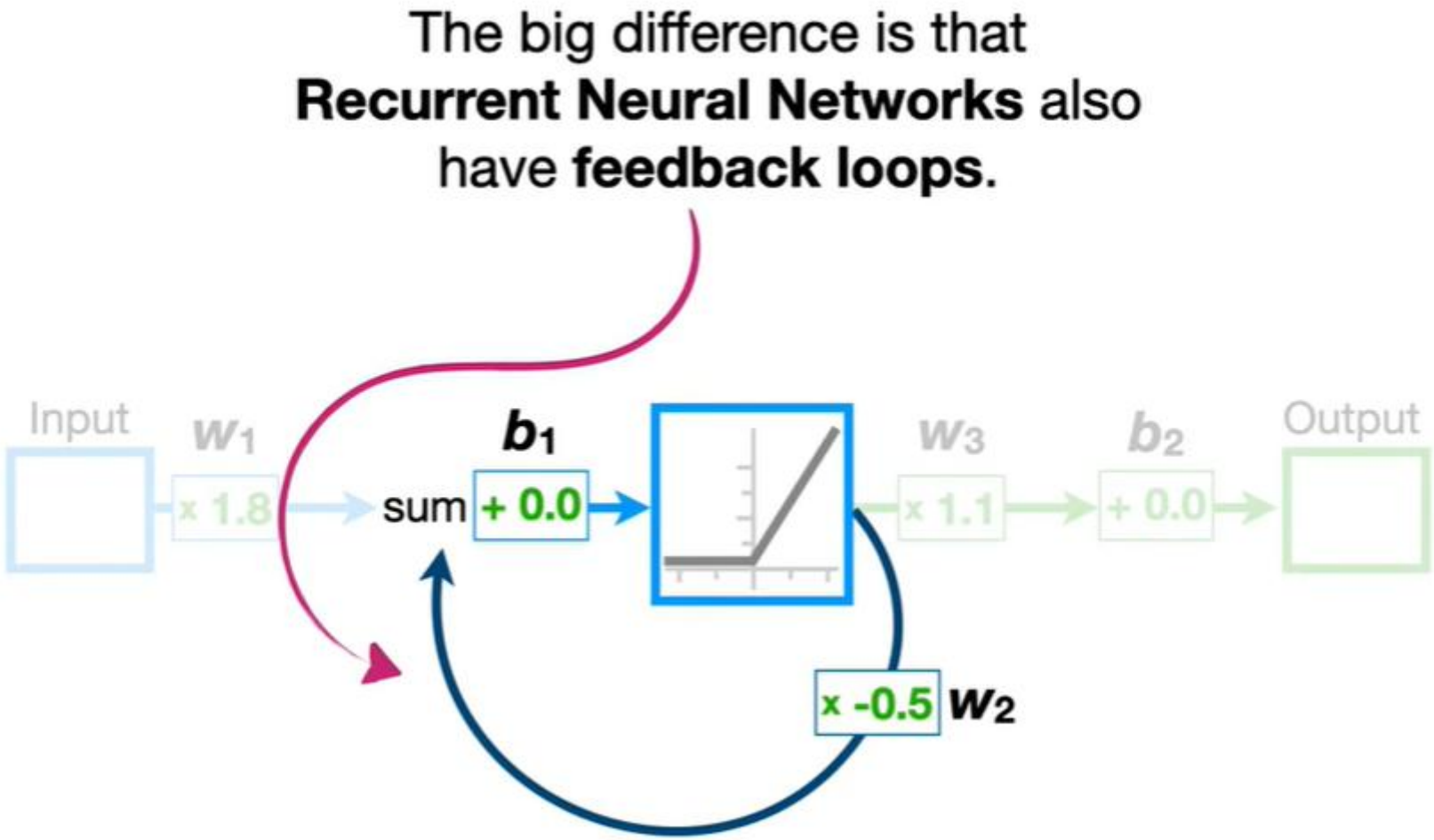
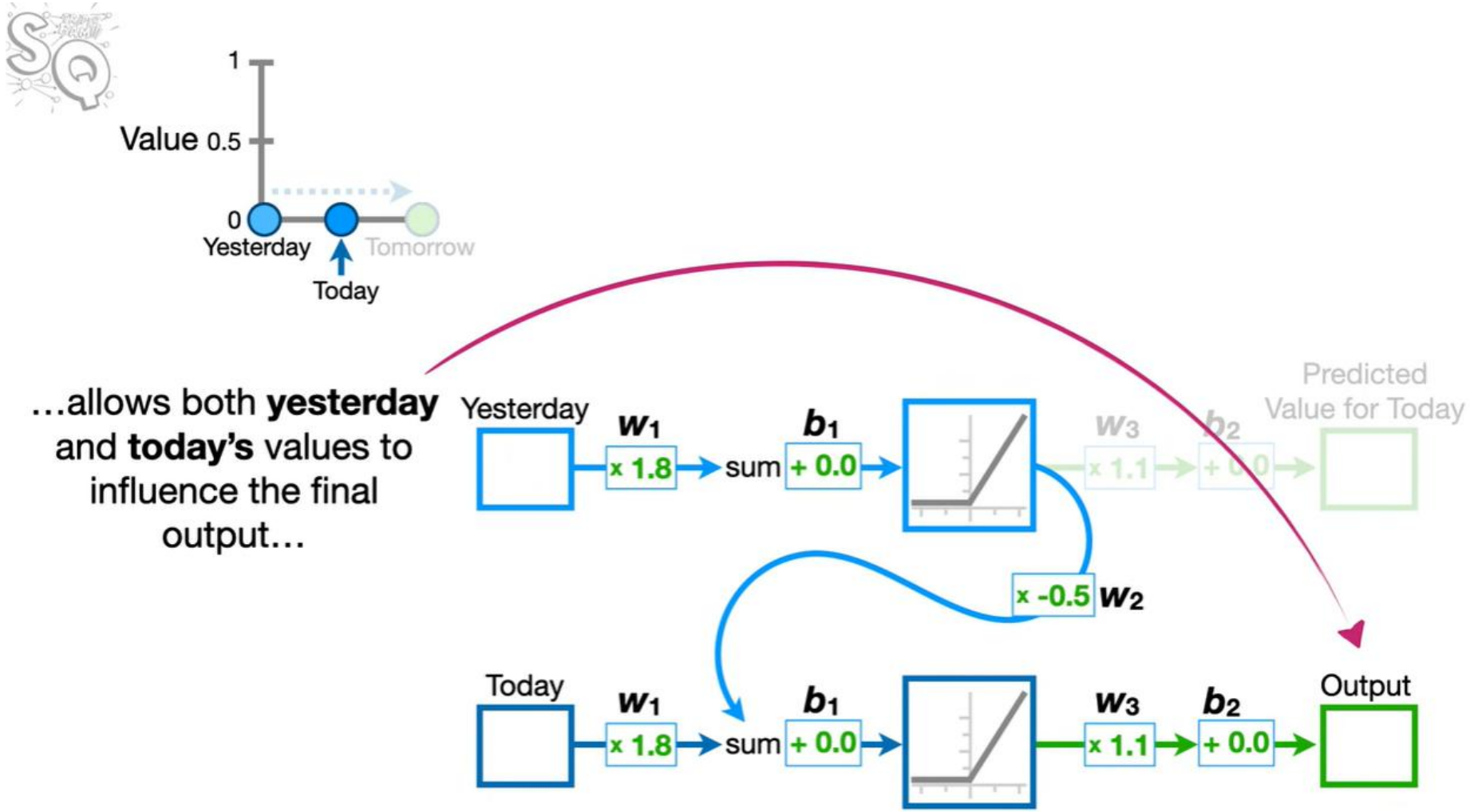
Advanced Neural Networks

Dr Chathura Rajapakse

Recurrent Neural Network

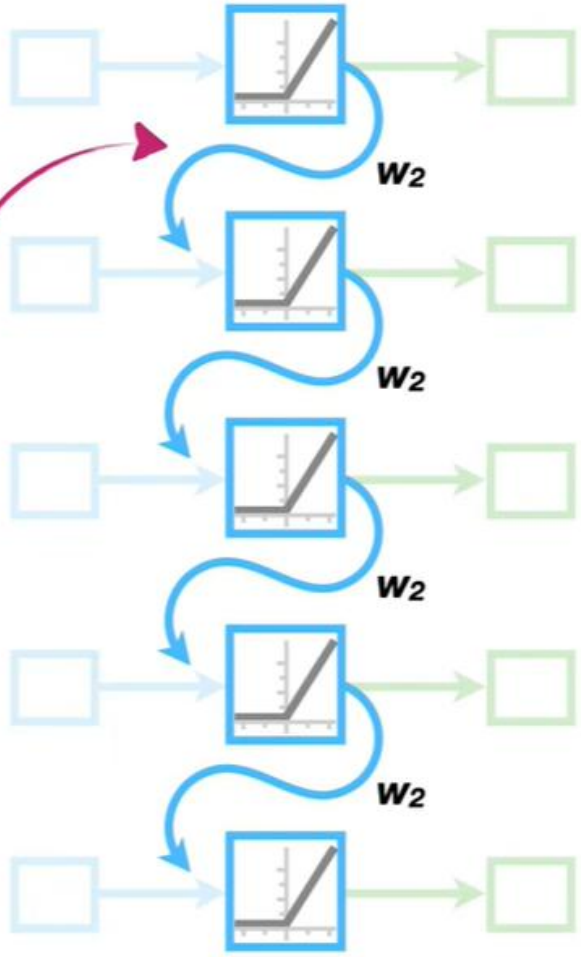


Recurrent Neural Network

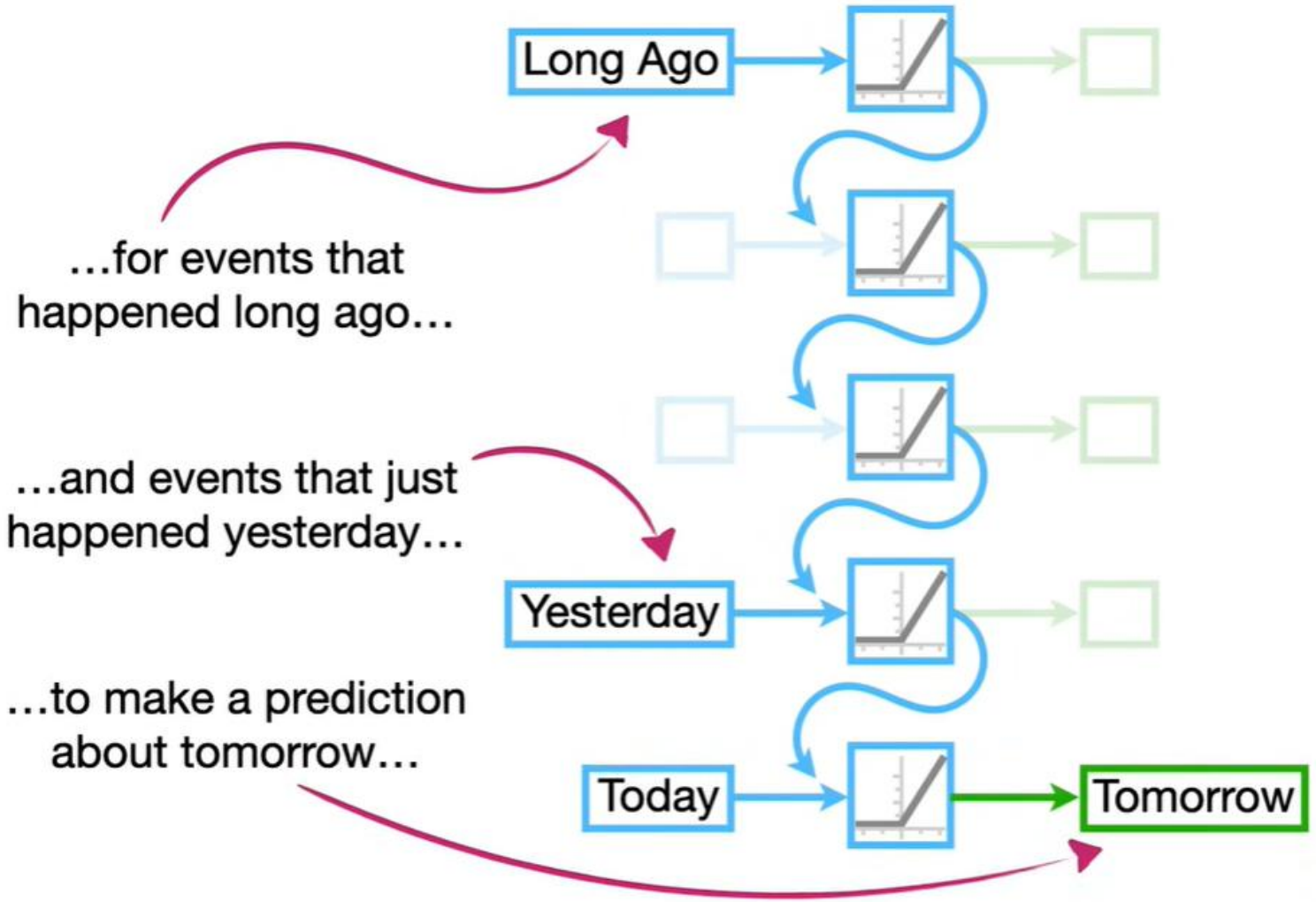


Vanishing/Exploding Gradient

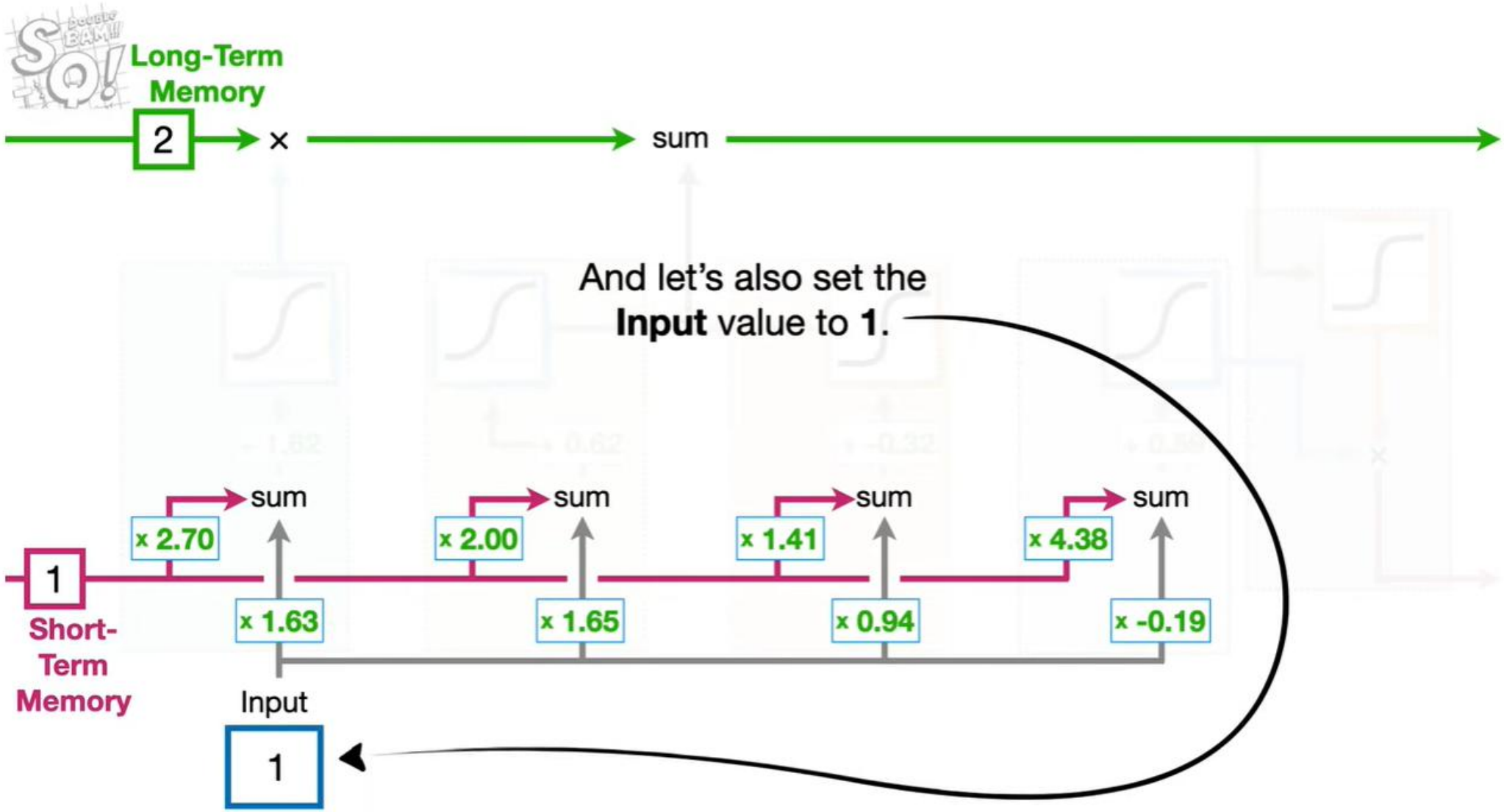
In our example, the gradient will ***explode***, when we set w_2 to any value larger than **1**.



Towards LSTM

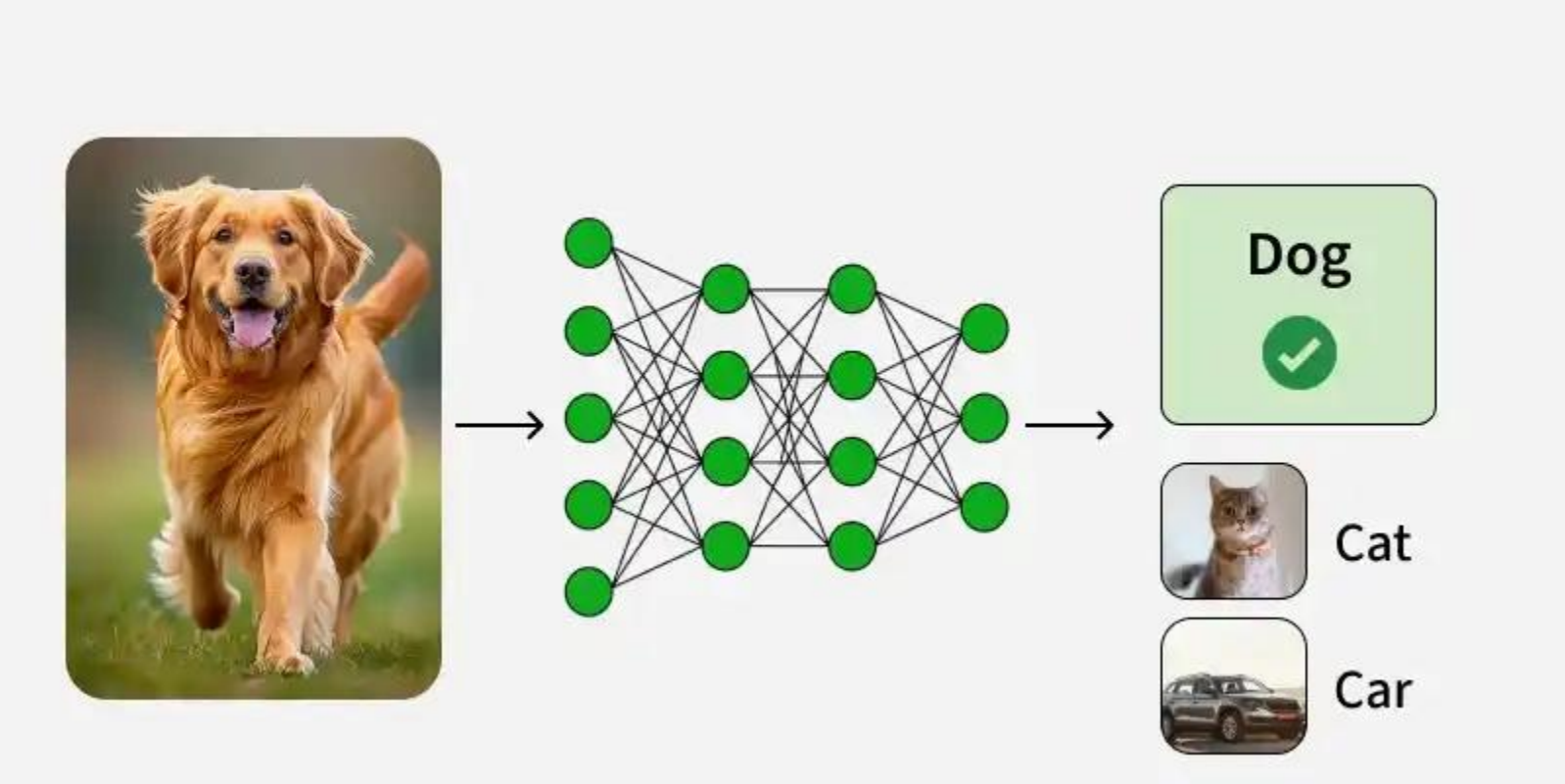


LSTM Basics


Long-Term Memory
2
×
sum
And let's also set the **Input** value to **1**.
sum
sum
sum
sum
x 2.70
x 2.00
x 1.41
x 4.38
1
x 1.63
x 1.65
x 0.94
x -0.19
Short-Term Memory
Input
1

Computers see the world

- Image recognition (classification)



Object Detection

- Like image classification, object detection attempts to predict the contents of an image, but now with multiple objects, not just a single one

- The models are trained to predict the “Bounding Boxes” of objects in the image, and apply labels for the contents of each box

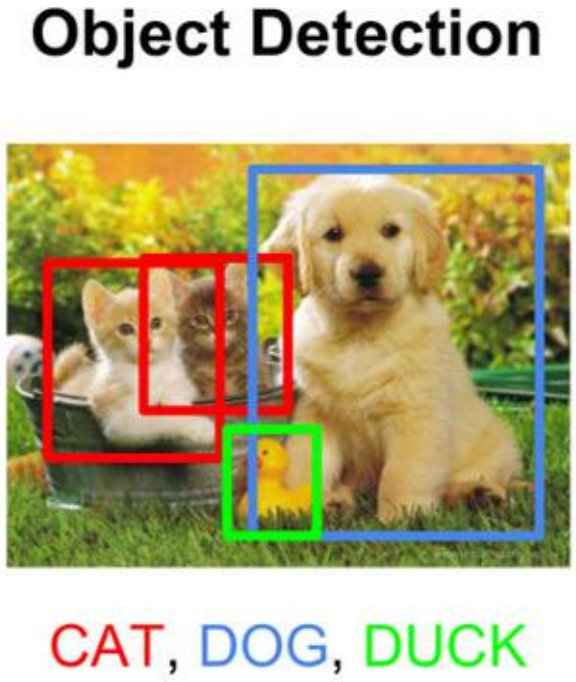
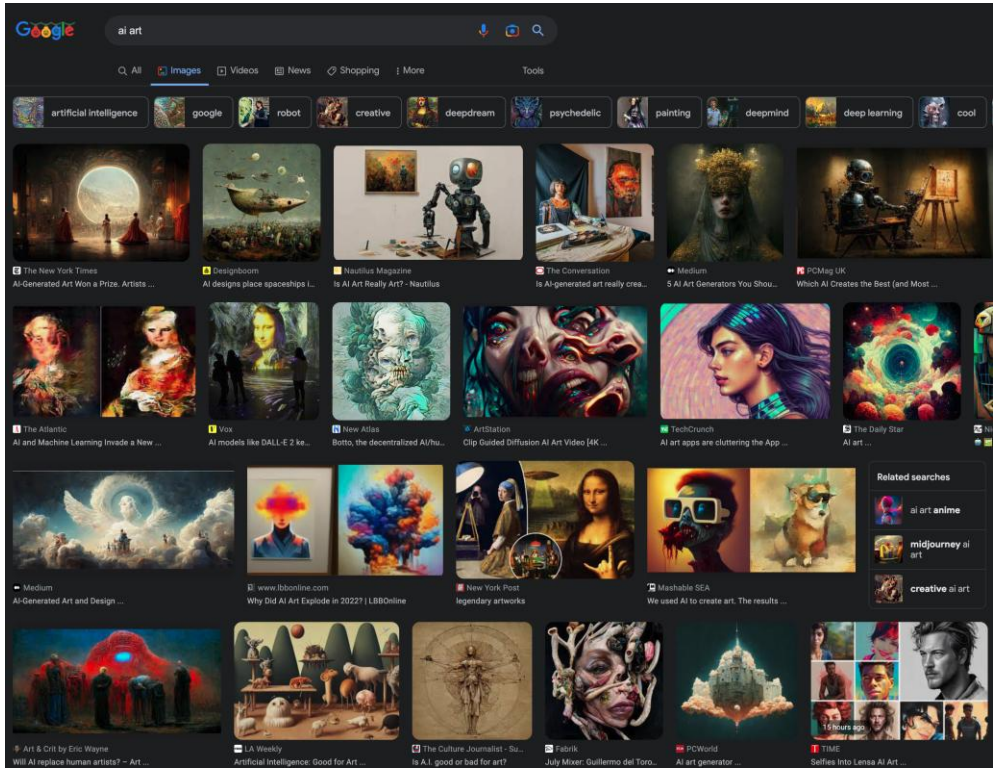
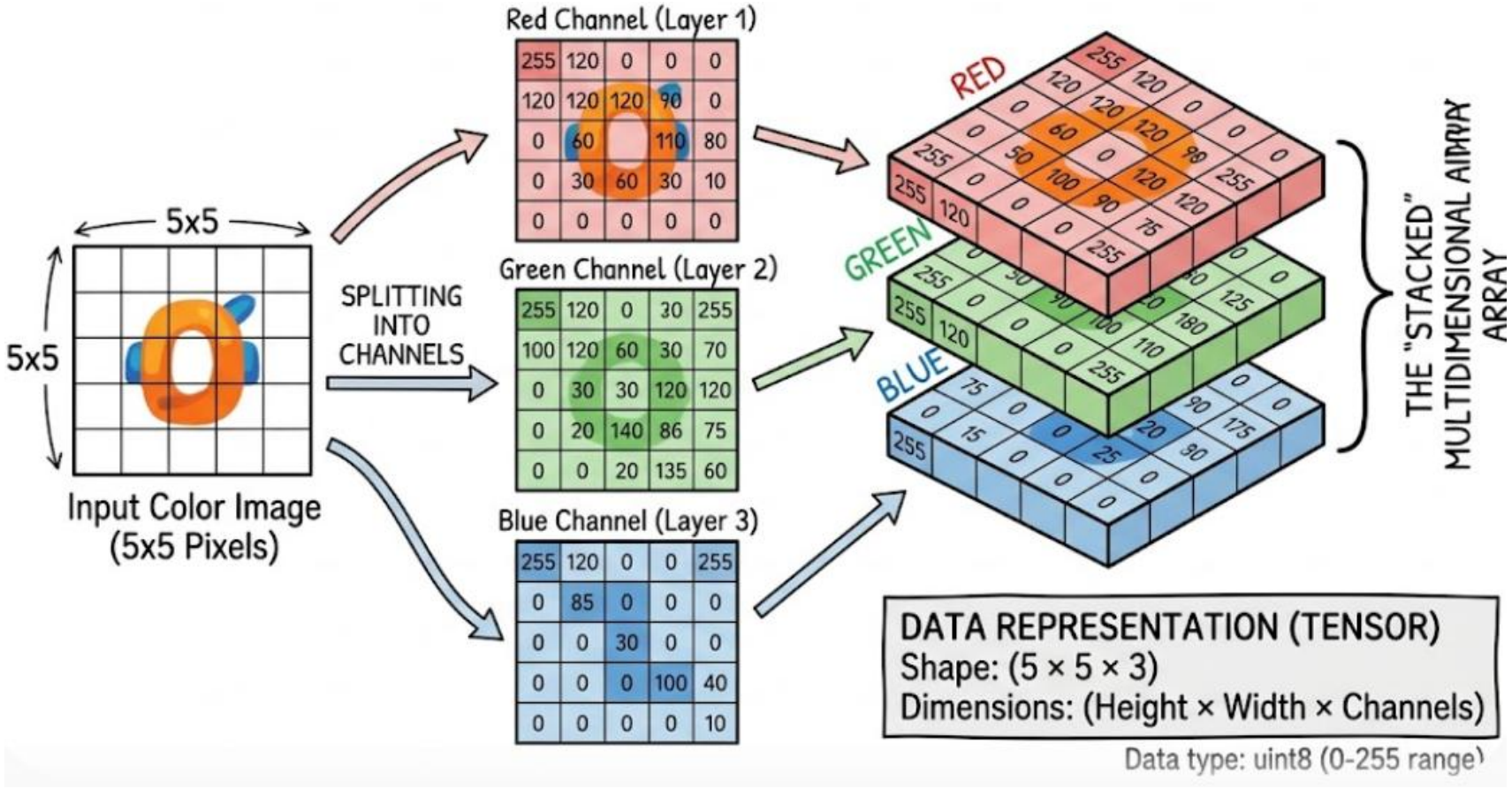


Image Generation

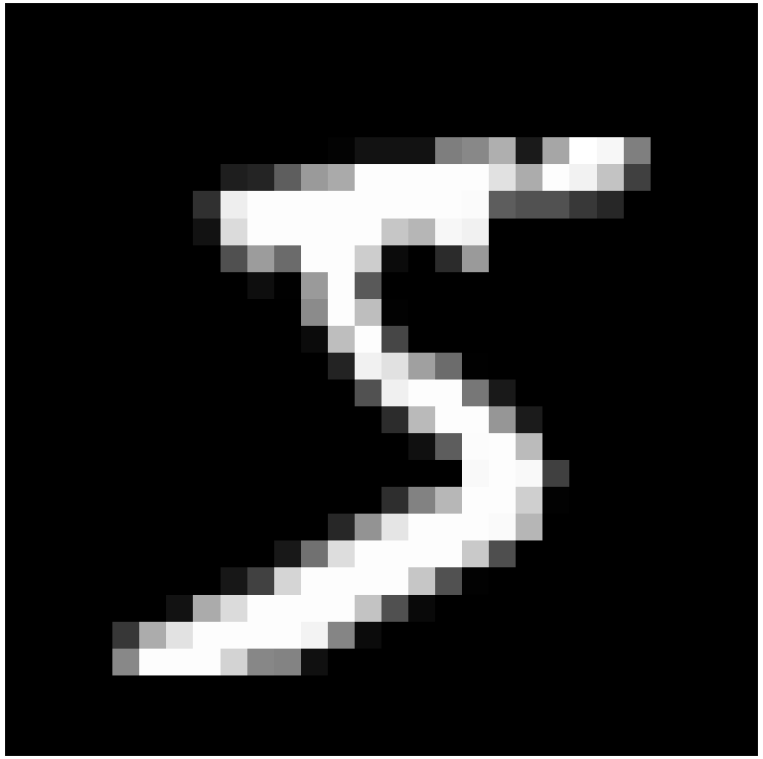
- Increasingly popular in the last few years, training models to create images, either from scratch or with a prompt, has many applications.
- Raise of Stable Diffusion has lead to an explosion of “AI Art”, but other models include VAEs and Generalised Adversarial Nets (GANs)
- Can be used for image restoration and scene reconstruction.

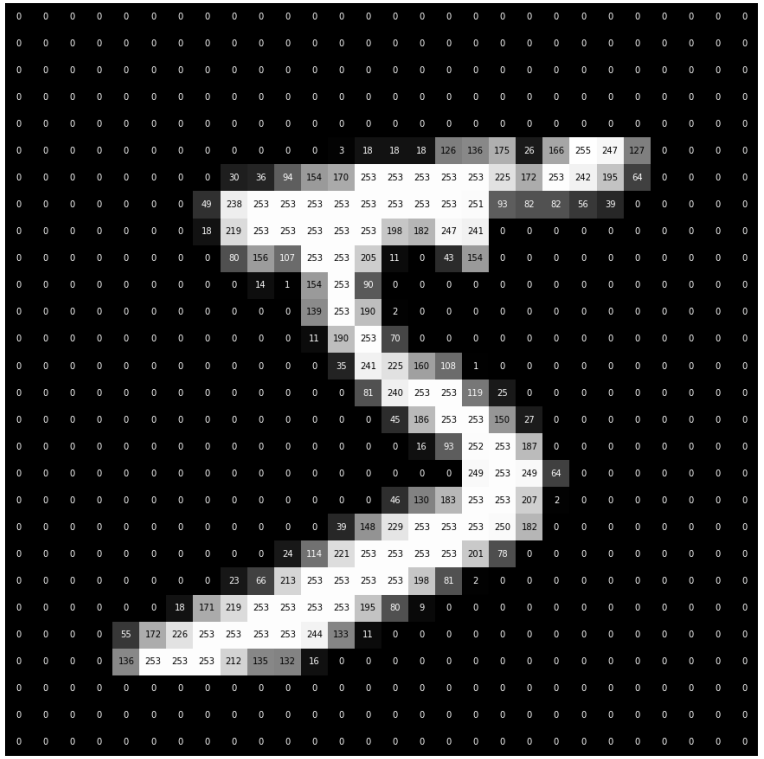


How do we set the features of an image?

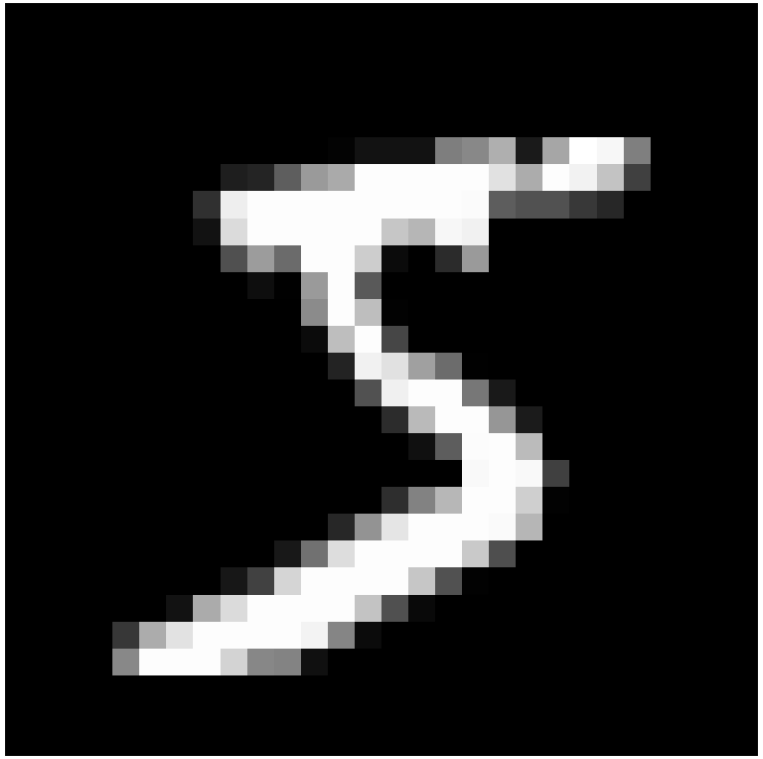


What do computers see?





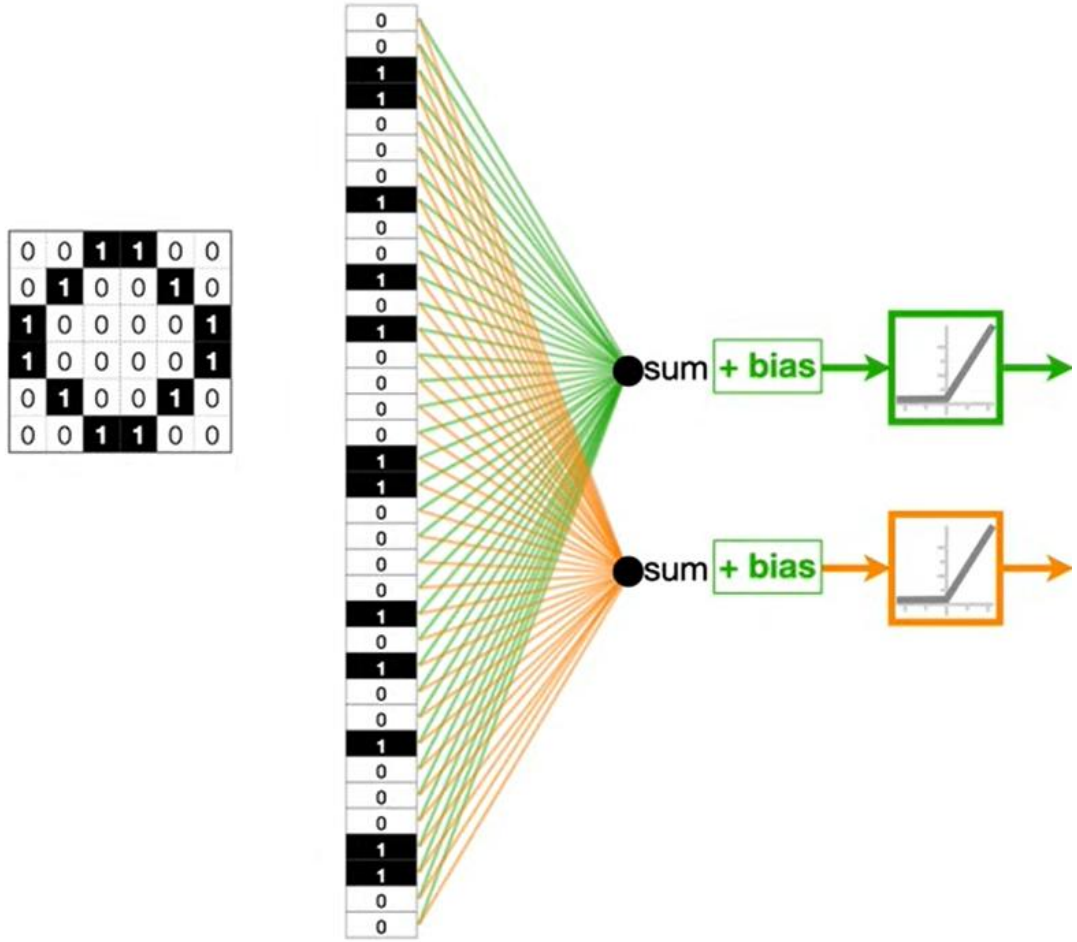
What do computers see?



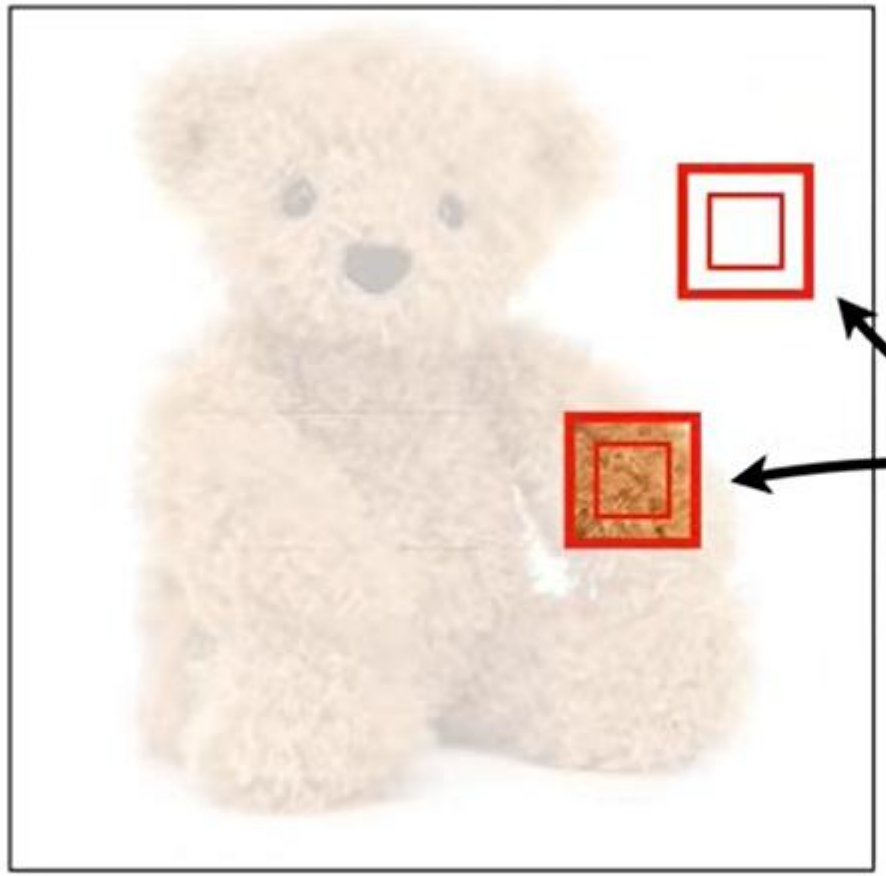
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	3	18	18	18	126	136	175	26	166	255	247	127	0	0	0	0
0	0	0	0	0	0	0	0	30	36	94	154	170	253	253	253	253	253	225	172	253	242	195	64	0	0	0	0
0	0	0	0	0	0	0	49	238	253	253	253	253	253	253	253	253	251	93	82	82	56	39	0	0	0	0	0
0	0	0	0	0	0	0	18	219	253	253	253	253	253	198	182	247	241	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	80	156	107	253	253	205	11	0	43	154	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	14	1	154	253	90	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	139	253	190	2	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	11	190	253	70	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	35	241	225	160	108	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	81	240	253	253	119	25	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	45	186	253	253	150	27	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	16	93	252	253	187	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	249	253	249	64	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	46	130	183	253	253	207	2	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	39	148	229	253	253	253	250	182	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	24	114	221	253	253	253	253	201	78	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	23	66	213	253	253	253	253	198	81	2	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	18	171	219	253	253	253	253	195	80	9	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	55	172	226	253	253	253	253	244	133	11	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	136	253	253	253	212	135	132	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Why standard NNs fail on images

- **Destroys Spatial Geometry:** Pixel neighborhood relationships are lost during 1D unrolling.
- **Parameter Explosion:** High-resolution inputs create millions of dense weight connections.
- **No Shift Invariance:** Small shifts in object location break model recognition.



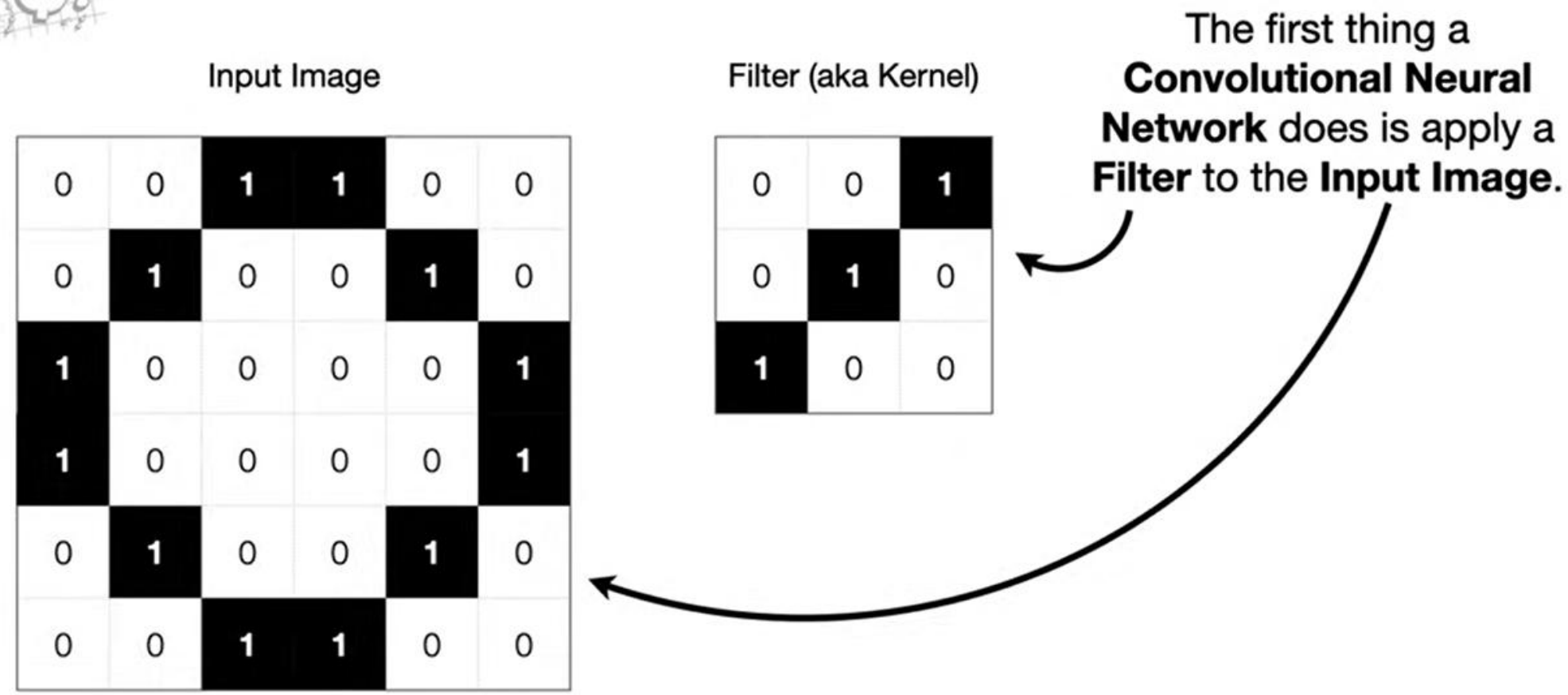
What CNNs does



1) Reduce the number of input nodes.
2) Tolerate small shifts in where the pixels are in the image.
3) Take advantage of the correlations that we observe in complex images.

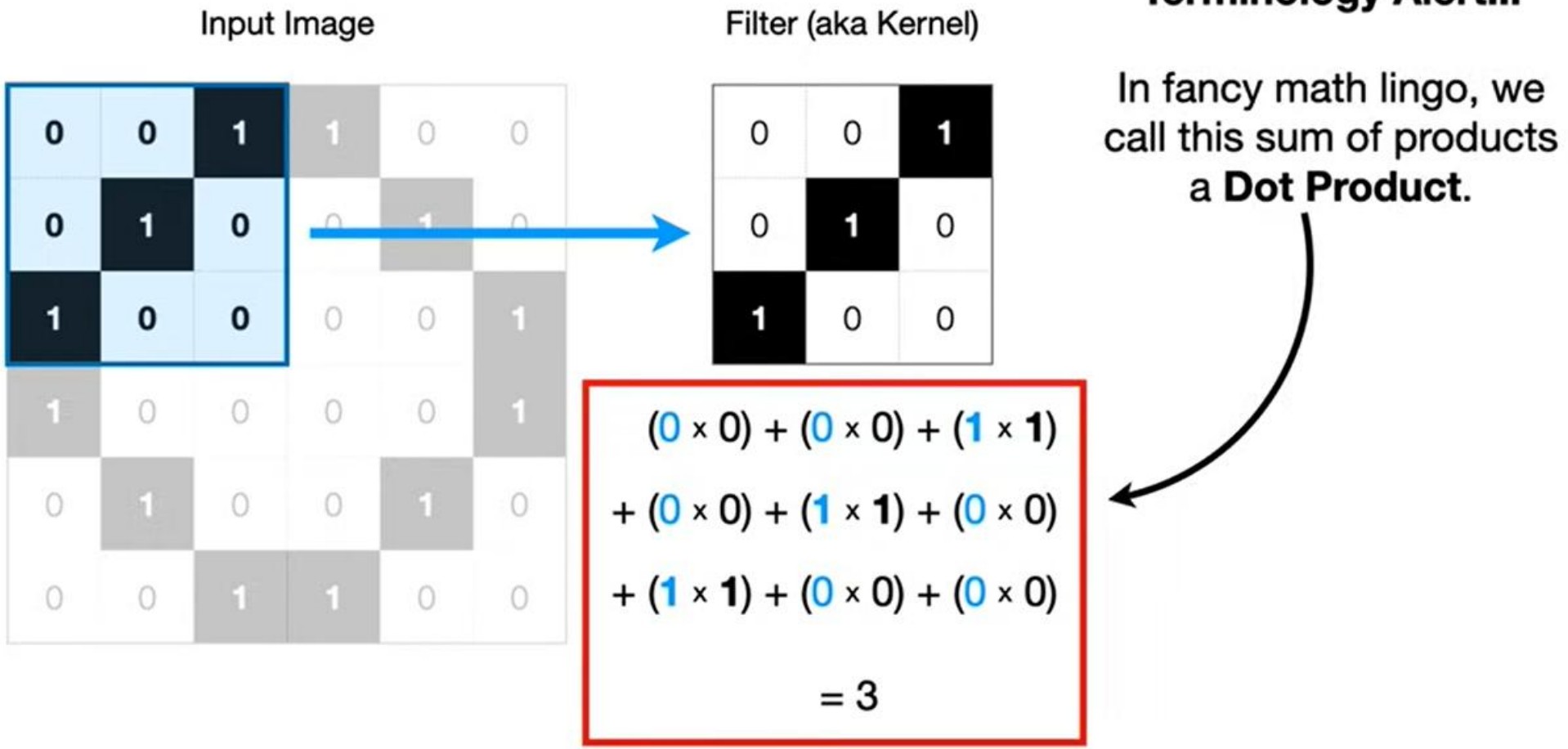
CNN – Step 01




Input Image
Filter (aka Kernel)
The first thing a **Convolutional Neural Network** does is apply a **Filter** to the **Input Image**.
0 0 1 1 0 0
0 1 0 0 1 0
1 0 0 0 0 1
1 0 0 0 0 1
0 1 0 0 1 0
0 0 1 1 0 0
0 0 1
0 1 0
1 0 0

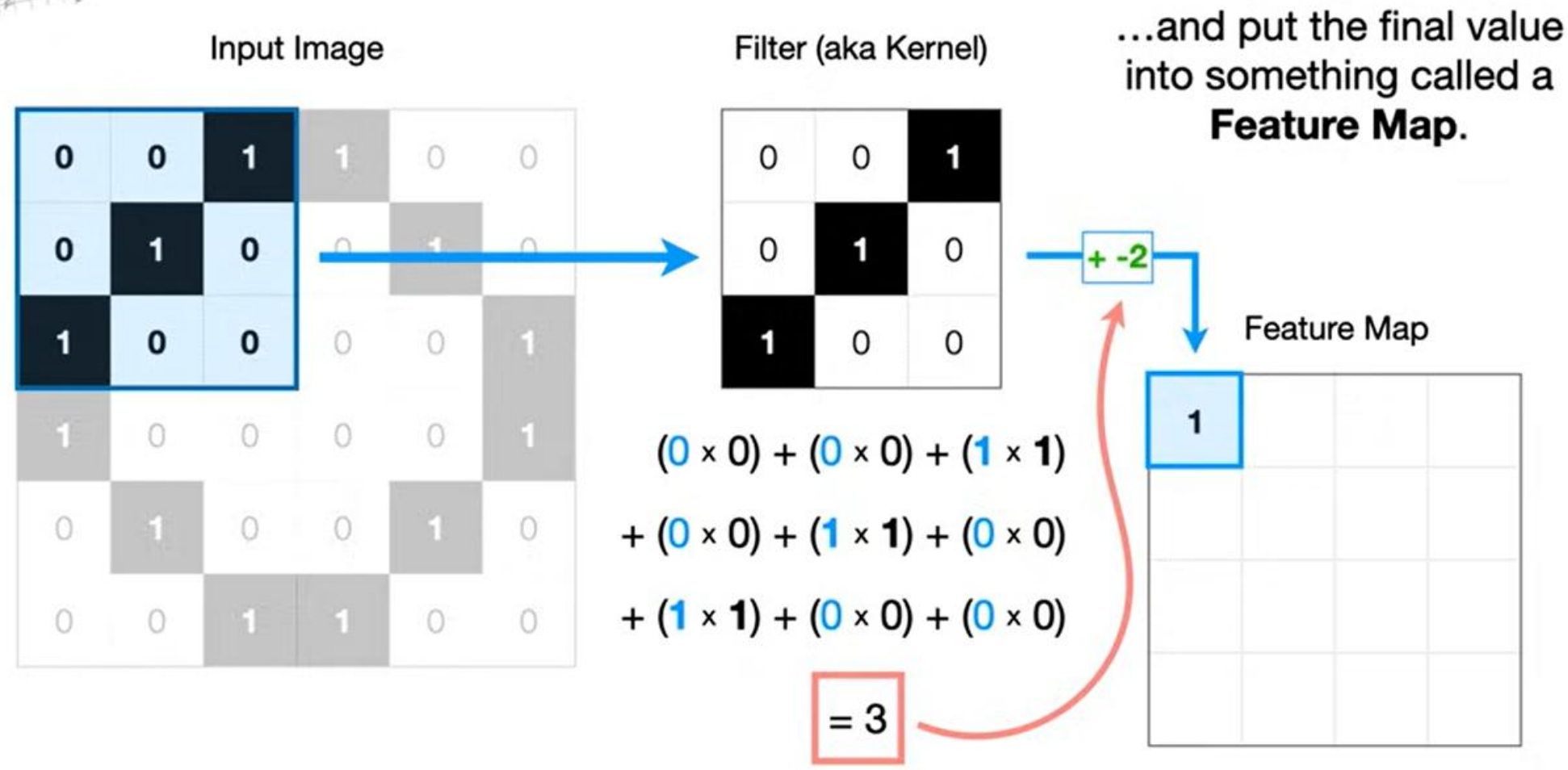
CNN – Step 02



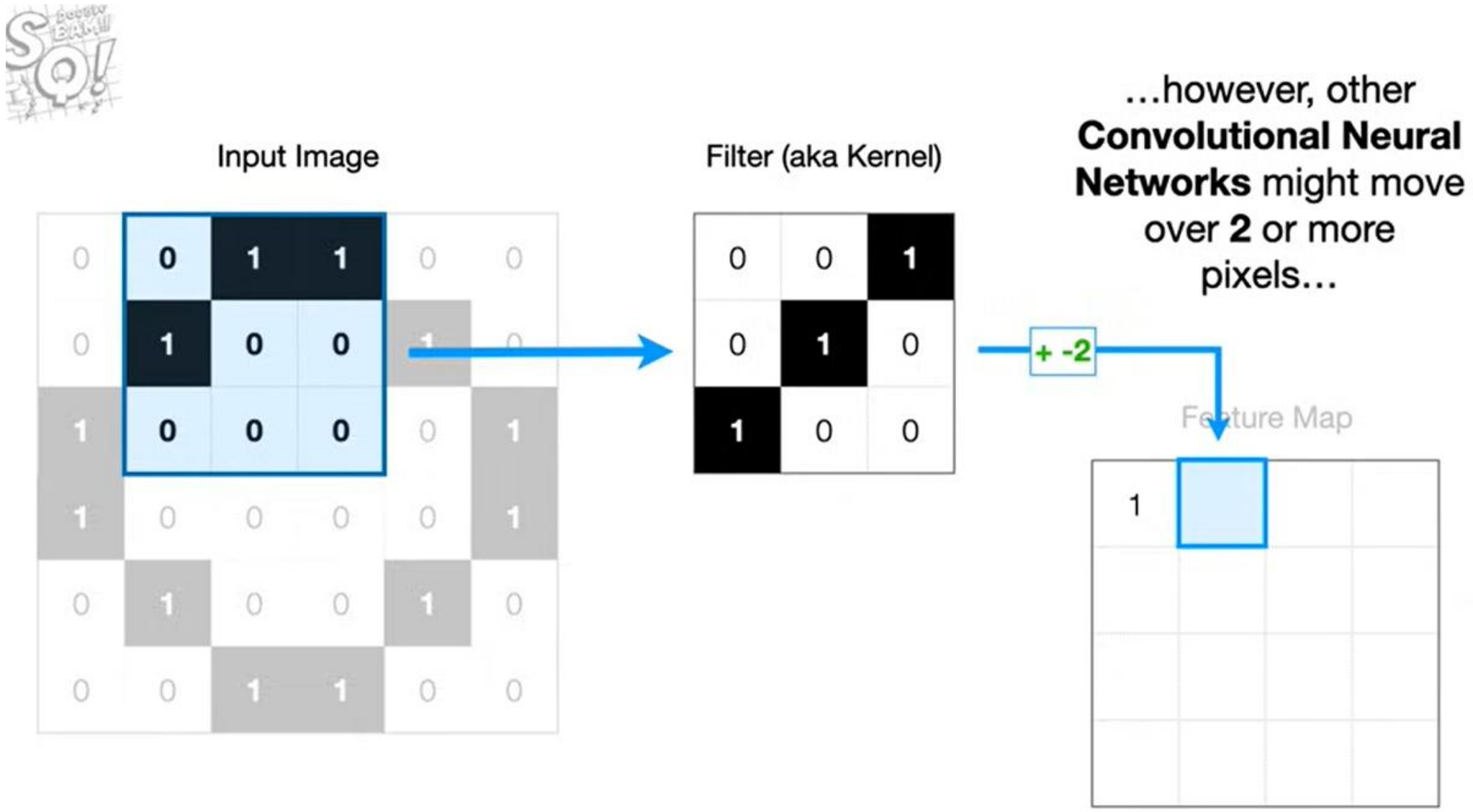

Input Image
Filter (aka Kernel)
0 0 1 1 0 0
0 1 0 0 1 0
1 0 0 0 0 1
1 0 0 0 0 1
0 1 0 0 1 0
0 0 1 1 0 0
0 0 1
0 1 0
1 0 0
(0 × 0) + (0 × 0) + (1 × 1)
+ (0 × 0) + (1 × 1) + (0 × 0)
+ (1 × 1) + (0 × 0) + (0 × 0)
= 3
Terminology Alert!!!
In fancy math lingo, we call this sum of products a Dot Product.

CNN – Step 03

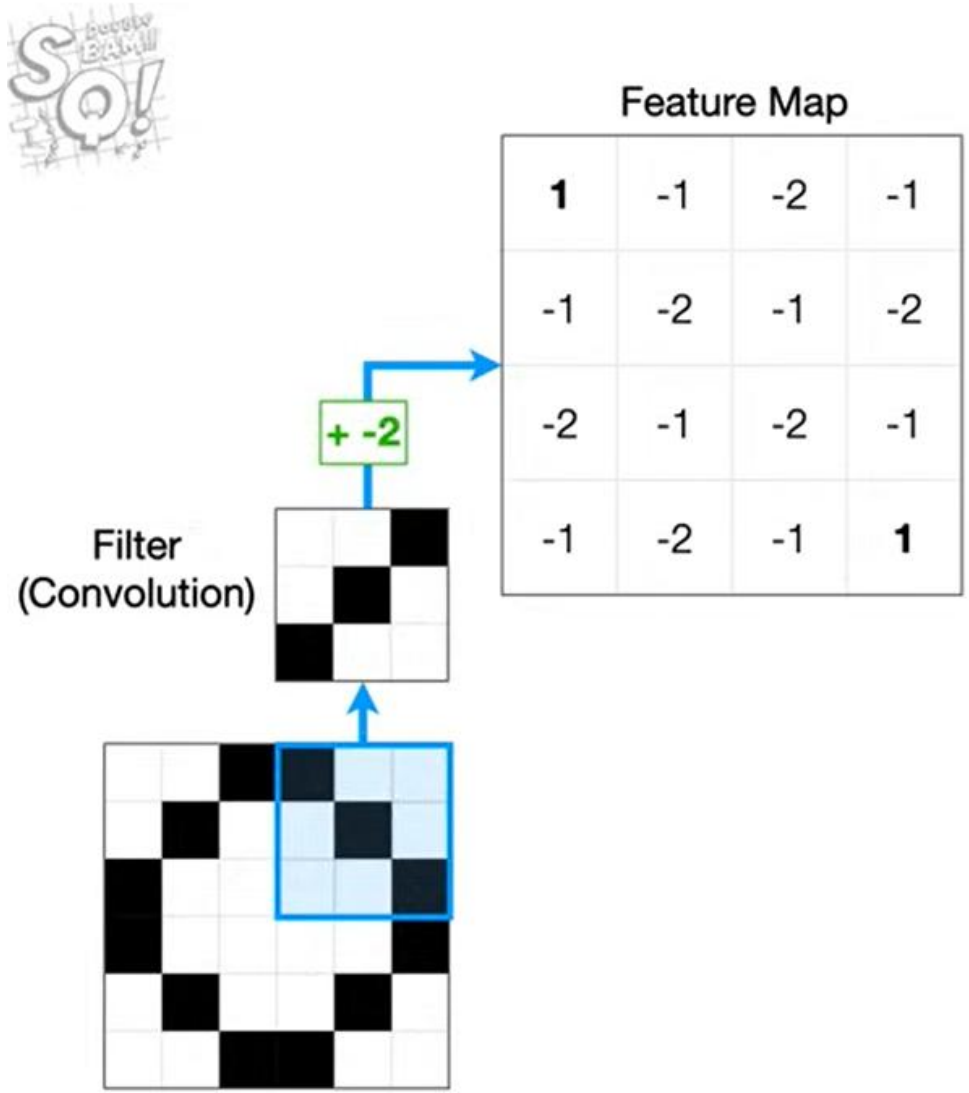


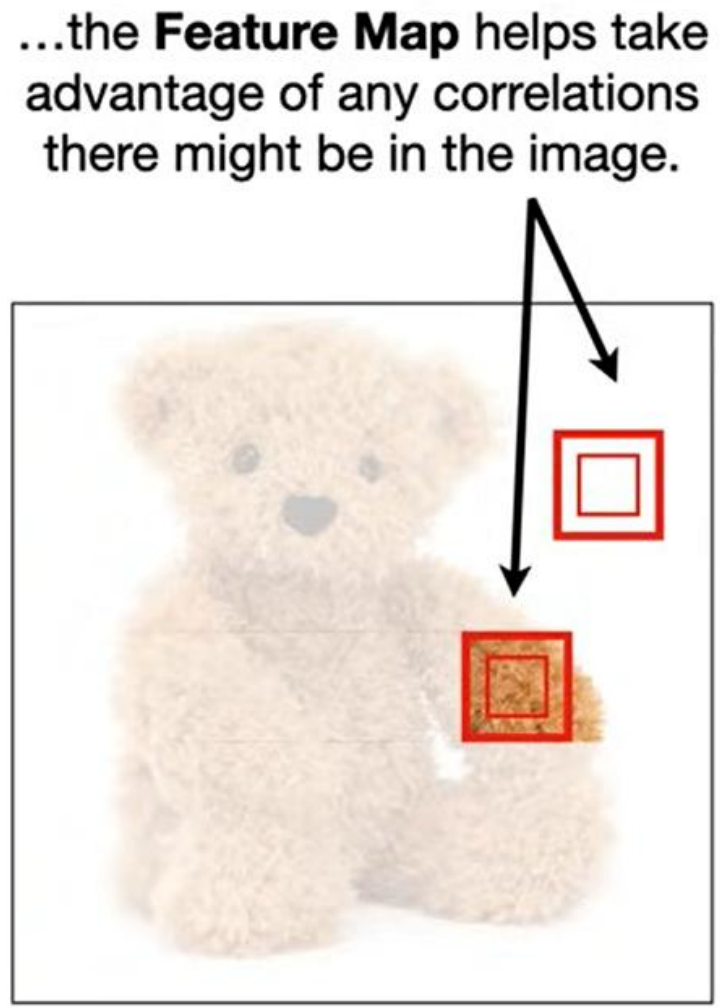

Input Image
Filter (aka Kernel)
…and put the final value into something called a **Feature Map**.
+ -2
Feature Map
(0 × 0) + (0 × 0) + (1 × 1)
+ (0 × 0) + (1 × 1) + (0 × 0)
+ (1 × 1) + (0 × 0) + (0 × 0)
= 3

CNN – Step 04

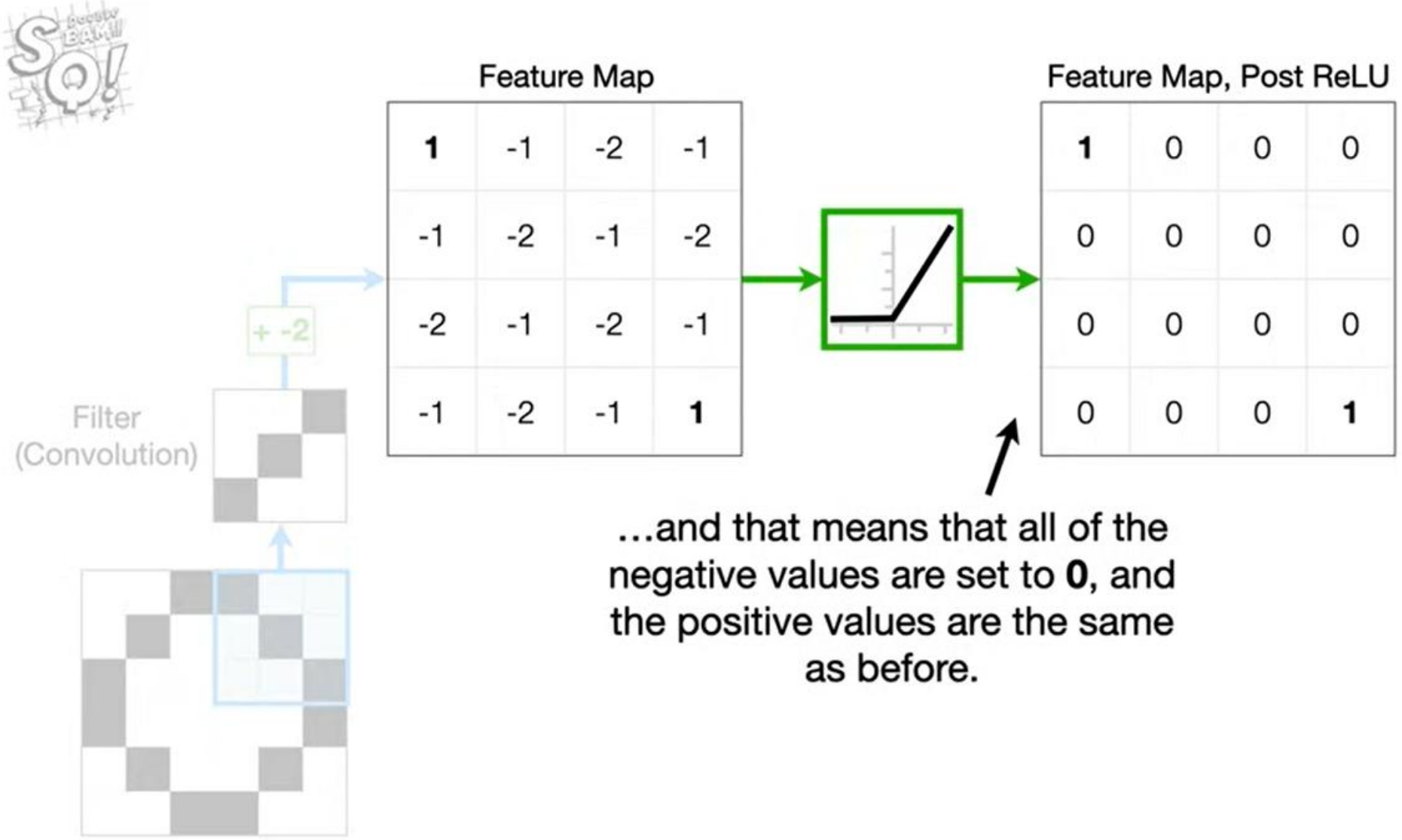

Input Image
Filter (aka Kernel)
…however, other **Convolutional Neural Networks** might move over **2** or more pixels…
+ -2
Feature Map

Why Feature Map

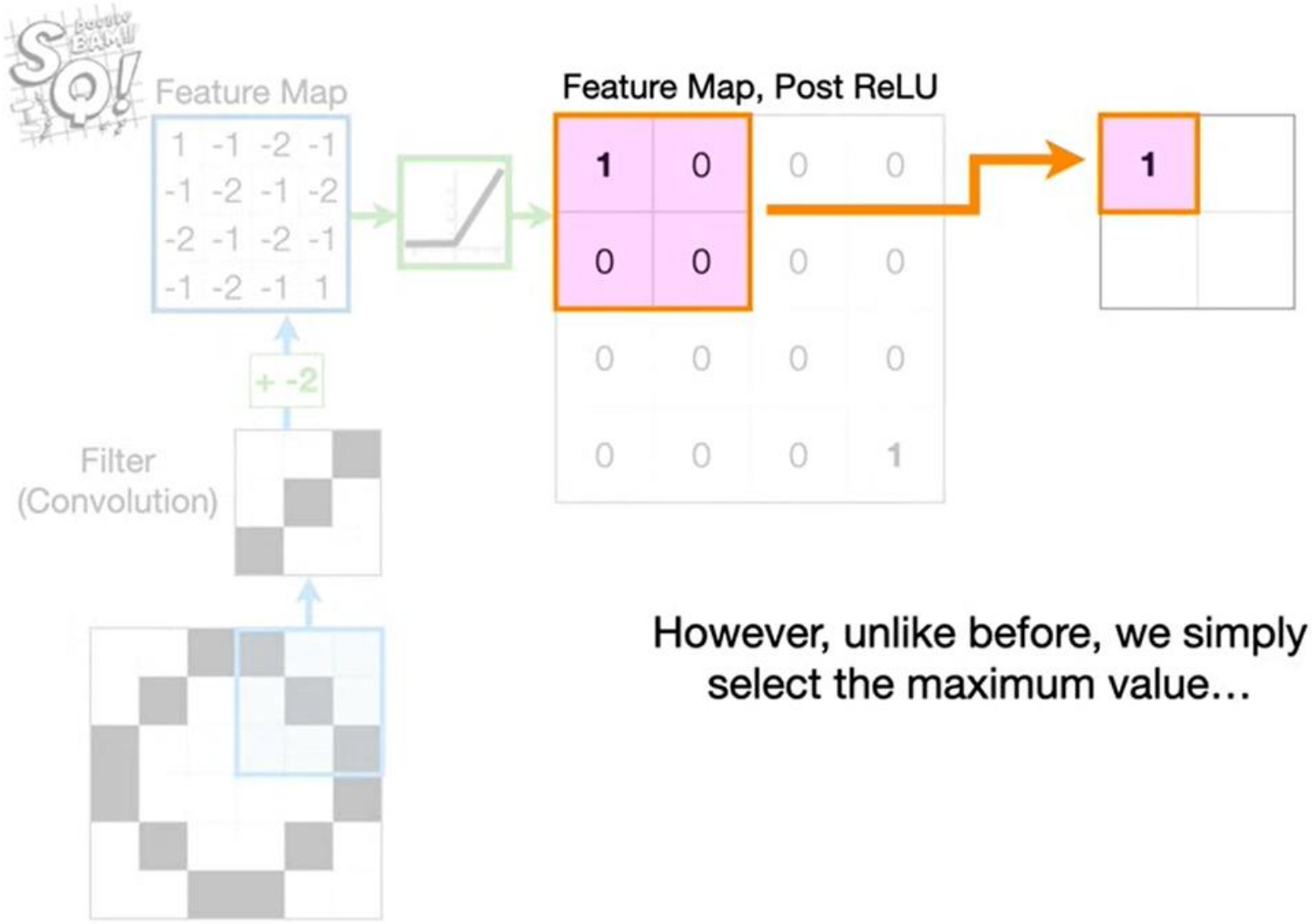




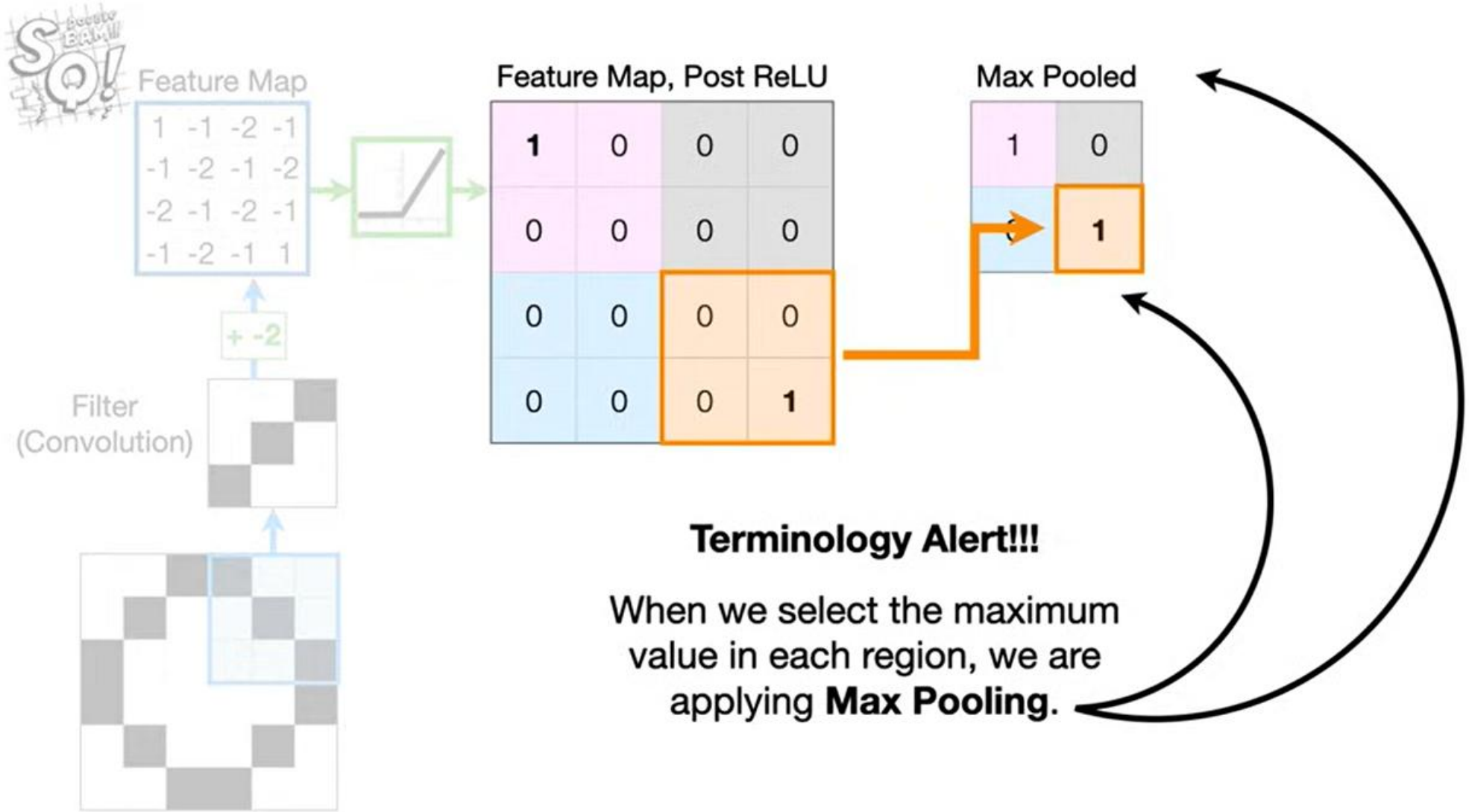
CNN – Step 05


Feature Map
1 -1 -2 -1
-1 -2 -1 -2
-2 -1 -2 -1
-1 -2 -1 1
Feature Map, Post ReLU
1 0 0 0
0 0 0 0
0 0 0 0
0 0 0 1
+ -2
Filter
(Convolution)
…and that means that all of the negative values are set to 0, and the positive values are the same as before.

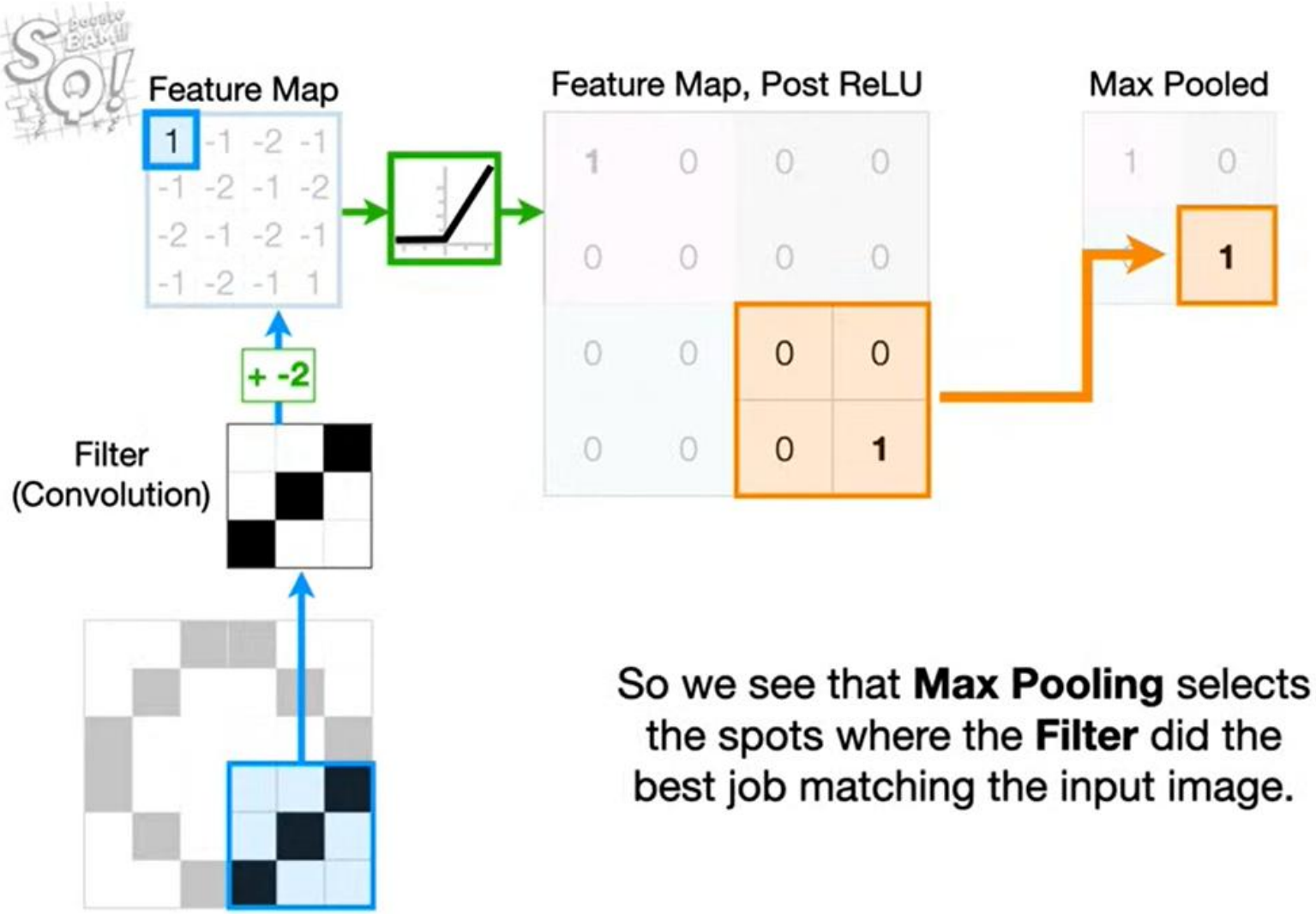
CNN – Step 06


Feature Map
1 -1 -2 -1
-1 -2 -1 -2
-2 -1 -2 -1
-1 -2 -1 1
+ -2
Filter
(Convolution)
Feature Map, Post ReLU
1 0 0 0
0 0 0 0
0 0 0 0
0 0 0 1
1
However, unlike before, we simply
select the maximum value…

Max pooling

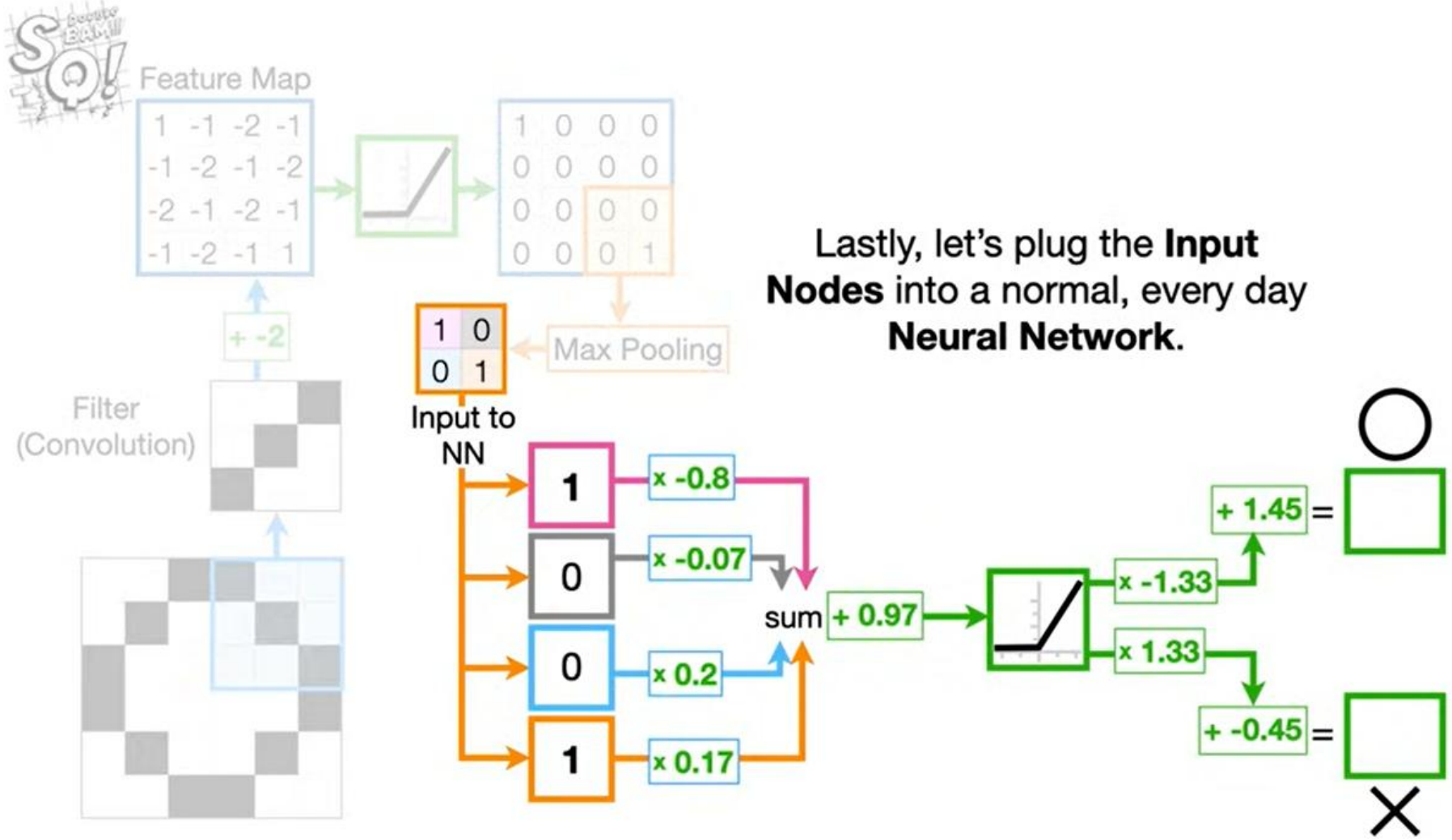

Feature Map
1 -1 -2 -1
-1 -2 -1 -2
-2 -1 -2 -1
-1 -2 -1 1
+ -2
Filter
(Convolution)
Feature Map, Post ReLU
1 0 0 0
0 0 0 0
0 0 0 0
0 0 0 1
Max Pooled
1 0
1
Terminology Alert!!!
When we select the maximum value in each region, we are applying Max Pooling.

Max pooling

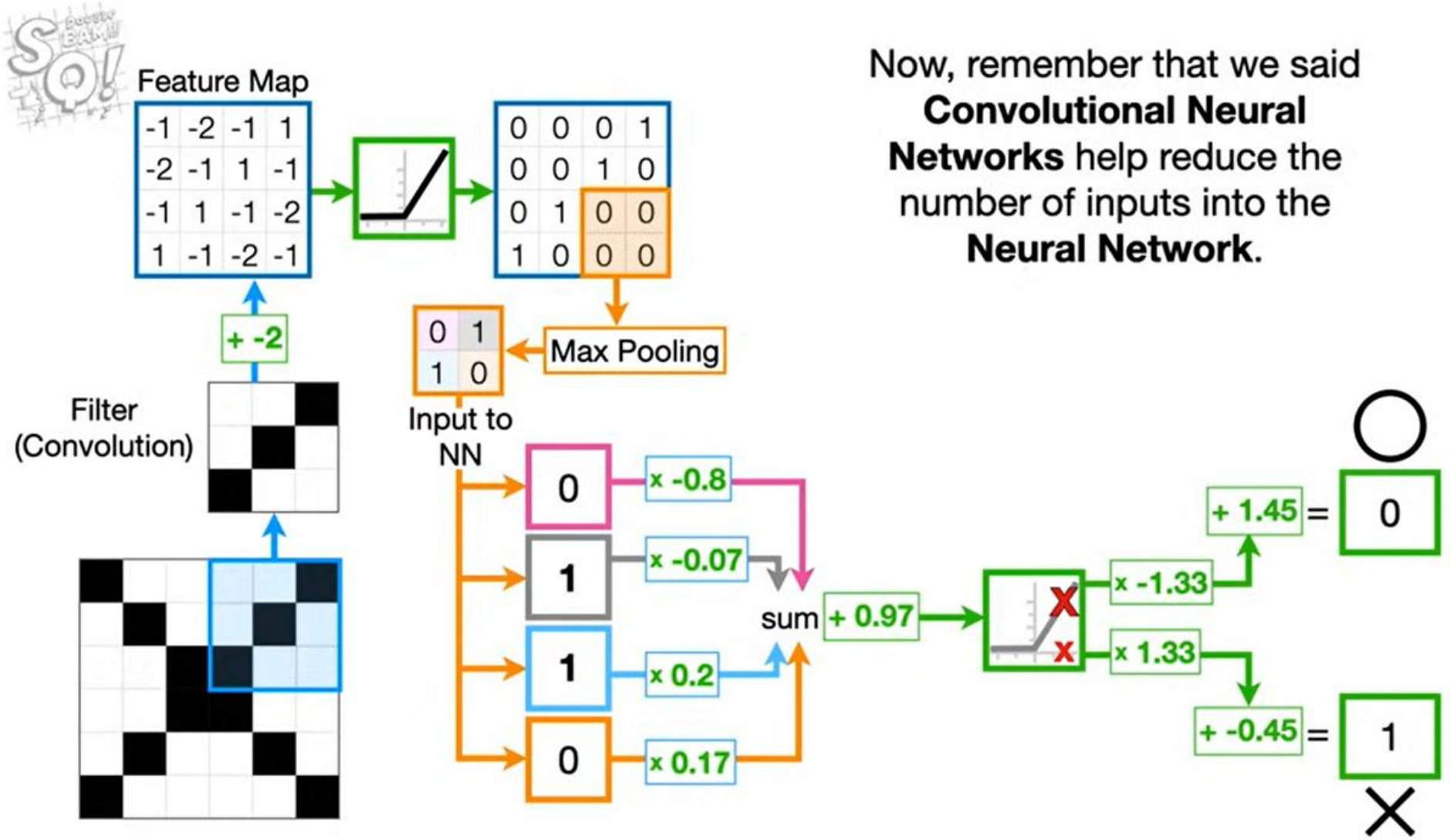


So we see that **Max Pooling** selects the spots where the **Filter** did the best job matching the input image.

CNN – Step 07


So!
Feature Map
1 -1 -2 -1
-1 -2 -1 -2
-2 -1 -2 -1
-1 -2 -1 1
+ -2
Filter
(Convolution)
1 0 0 0
0 0 0 0
0 0 0 0
0 0 0 1
Max Pooling
1 0
0 1
Input to
NN
Lastly, let's plug the **Input Nodes** into a normal, every day **Neural Network.**
1
0
0
1
x -0.8
x -0.07
x 0.2
x 0.17
sum
+ 0.97
x -1.33
x 1.33
+ 1.45 =
+ -0.45 =

CNN – Step 08


Feature Map
-1 -2 -1 1
-2 -1 1 -1
-1 1 -1 -2
1 -1 -2 -1
+ -2
Filter
(Convolution)
0 0 0 1
0 0 1 0
0 1 0 0
1 0 0 0
Max Pooling
0 1
1 0
Input to
NN
0
1
1
0
x -0.8
x -0.07
x 0.2
x 0.17
sum
+ 0.97
x -1.33
x 1.33
+ 1.45 =
0
+ -0.45 =
1
Now, remember that we said **Convolutional Neural Networks** help reduce the number of inputs into the **Neural Network**.

CNN Code

- Interpret this

```
import torch.nn as nn

# Complete CNN Architecture for a 32x32 input image (e.g., CIFAR-10)
model = nn.Sequential(
    # ==================== FEATURE EXTRACTION ====================
    # Layer 1: Input (3, 32, 32) -> Conv -> (32, 30, 30) -> MaxPool -> (32, 15, 15)
    nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),

    # Layer 2: Input (32, 15, 15) -> Conv -> (64, 13, 13) -> MaxPool -> (64, 6, 6)
    nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),

    # Layer 3: Input (64, 6, 6) -> Conv -> (128, 4, 4) -> MaxPool -> (128, 2, 2)
    nn.Conv2d(in_channels=64, out_channels=128, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),  # Optional pooling to downsample spatial size furt

    # ==================== FLATTENING STEP ====================
    # Reshapes 3D tensor (128 channels x 2 height x 2 width) into a 1D vector of siz
    nn.Flatten(),

    # ==================== CLASSIFICATION (FFNN) ====================
    # Hidden Layer: 512 inputs -> 128 hidden units
    nn.Linear(in_features=128 * 2 * 2, out_features=128),
    nn.ReLU(),

    # Output Layer: 128 hidden units -> 10 class logits (e.g., dog, cat, car, etc.)
    nn.Linear(in_features=128, out_features=10)
)
```

Assignment 02

For this assignment, you will build and train a PyTorch U-Net autoencoder on Google Colab to automatically colorize grayscale images using the Lab color space. Your task is to implement a data pipeline that resizes images to 128×128 pixels, converts them from RGB to Lab space, passes the single-channel L (lightness) as input, and trains the network to predict the two-channel a and b color components using Mean Squared Error (MSE) loss. Once your model is trained for 15–20 epochs, you will apply it to a personal historical black-and-white photograph (such as one of your grandparents) by extracting its L channel, predicting its missing a and b color channels, and recombining all three back into a full RGB image. You will submit a single Notebook containing your dataset code, U-Net architecture, training metrics, and a final figure displaying the grayscale input, model prediction, and fully restored color image.